



Architecture des ordinateurs

Chapitre 7

Programmation en assembleur



CH07 - Programmation en assembleur

7.1 INTRODUCTION

7.2 CARACTÉRISTIQUES DU PROCESSEUR MIPS

7.3 REGISTRES DU PROCESSEUR

7.4 FAMILLES D'INSTRUCTIONS

7.5 MODES D'ADRESSAGE

7.6 APPELS DE SOUS-PROGRAMMES

7.7 APPELS SYSTÈMES

7.8 DIRECTIVES

7.9 EXEMPLES DE PROGRAMMES



7.1 Introduction

- Le **langage d'assemblage** représente de manière **symbolique** les instructions du processeur
- Objectif : mieux comprendre comment un programme écrit dans un langage de haut-niveau est exécuté par le processeur
- **Assembleur MIPS** car jeu d'instruction réduit et homogène



CH07 - Programmation en assembleur

7.1 INTRODUCTION

7.2 CARACTÉRISTIQUES DU PROCESSEUR MIPS

7.3 REGISTRES DU PROCESSEUR

7.4 FAMILLES D'INSTRUCTIONS

7.5 MODES D'ADRESSAGE

7.6 APPELS DE SOUS-PROGRAMMES

7.7 APPELS SYSTÈMES

7.8 DIRECTIVES

7.9 EXEMPLES DE PROGRAMMES



7.2 Caractéristiques du processeur MIPS

Processeur MIPS = **architecture RISC**

- Bus d'adresse de 32 bits → mémoire max = 4 Go
- Mots mémoires de 32 bits (4 octets) **alignés** sur des adresses qui sont des multiples de 4
- Code machine d'une instruction sur 32 bits
- 32 registres généraux de 32 bits
- Valeurs entières signées en **complément à 2**
- Représentation **petit-boutiste** (little-endian)



CH07 - Programmation en assembleur

7.1 INTRODUCTION

7.2 CARACTÉRISTIQUES DU PROCESSEUR MIPS

7.3 REGISTRES DU PROCESSEUR

7.4 FAMILLES D'INSTRUCTIONS

7.5 MODES D'ADRESSAGE

7.6 APPELS DE SOUS-PROGRAMMES

7.7 APPELS SYSTÈMES

7.8 DIRECTIVES

7.9 EXEMPLES DE PROGRAMMES



7.3 Registres du processeur

NUM	NOM	USAGE
0	\$zero	Valeur constante 0
1	\$at	Temporaire réservé à l'assembleur
2-3	\$v0-\$v1	Valeurs de résultat (évaluation de fonctions ou d'expressions)
4-7	\$a0-\$a3	Arguments (appel de sous-programme)
8-15	\$t0-\$t7	Temporaires
16-23	\$s0-\$s7	Temporaires sauvegardés lors d'un appel de sous-programme
24-25	\$t8-\$t9	Temporaires
26-27	\$k0-\$k1	Réservé pour le noyau (Kernel)
28	\$gp	Pointeur global (<u>G</u> lobal <u>P</u> ointer)
29	\$sp	Pointeur de pile (<u>S</u> tack <u>P</u> ointer) [Gestion des sous-programmes]
30	\$fp	Pointeur de fenêtre (<u>F</u> rame <u>P</u> ointer) [Gestion des sous-programmes]
31	\$ra	Adresse de retour (<u>R</u> eturn <u>A</u> ddress) [Gestion des sous-programmes]

Utilisables dans les instructions et modifiables



7.3 Registres du processeur

Registres **internes** au CPU

- *Compteur ordinal*, **PC** (*Program Counter*)
 - **Adresse** de l'instruction courante
- *Registre d'instruction*, **IR** (*Instruction Register*)
 - **Code machine** de l'instruction courante

Ils ne sont pas accessibles au programme!



CH07 - Programmation en assembleur

7.1 INTRODUCTION

7.2 CARACTÉRISTIQUES DU PROCESSEUR MIPS

7.3 REGISTRES DU PROCESSEUR

7.4 FAMILLES D'INSTRUCTIONS

7.4.1 INSTRUCTIONS ARITHMÉTIQUES

7.4.2 INSTRUCTIONS LOGIQUES

7.4.3 INSTRUCTIONS DE DÉCALAGE

7.4.4 INSTRUCTIONS DE TRANSFERT AVEC LA MÉMOIRE

7.4.5 PSEUDO-INSTRUCTIONS

7.4.6 INSTRUCTIONS DE BRANCHEMENT

[...]



7.4 Familles d'instructions

7.4.1 Instructions arithmétiques

Instruction	Signification	Description
add \$s1, \$s2, \$s3	$\$s1 = \$s2 + \$s3$	Addition de 2 registres
addi \$s1, \$s2, 20	$\$s1 = \$s2 + 20$	Addition entre 1 registre et une constante
addu \$s1, \$s2, \$s3	$\$s1 = \$s2 + \$s3$	Addition non signée de 2 registres
addiu \$s1, \$s2, 20	$\$s1 = \$s2 + 20$	Addition non signée entre 1 registre et une constante
sub \$s1, \$s2, \$s3	$\$s1 = \$s2 - \$s3$	Soustraction de 2 registres
subu \$s1, \$s2, \$s3	$\$s1 = \$s2 - \$s3$	Soustraction non signée de 2 registres
div \$s1, \$s2	$Lo = \$s1 / \$s2, Hi = \$s1 \% \$s2$	Division entière de 2 registres, résultats dans {Hi,Lo}
divu \$s1, \$s2	$Lo = \$s1 / \$s2, Hi = \$s1 \% \$s2$	Division entière non signée de 2 registres, résultats dans {Hi,Lo}
mult \$s1, \$s2	$\{Hi, Lo\} = \$s1 * \$s2$	Multiplication de 2 registres, résultats dans {Hi,Lo}
multu \$s1, \$s2	$\{Hi, Lo\} = \$s1 * \$s2$	Multiplication non signée de 2 registres, résultats dans {Hi,Lo}

- Variantes : add, addi, addu, addiu
- Registre de résultat explicite ou implicite {Hi, Lo}



7.4.1 Instructions arithmétiques

Expressions complexes

- Langage de haut-niveau

$s = a + b + 100 - c$

- Langage d'assemblage

(Exemple avec s , a , b , c dans $\$t0$, $\$t1$, $\$t2$, $\$t3$)

`add $t0, $t1, $t2 # $t0 = $t1 + $t2 ->  $s = a + b$`

`addi $t0, $t0, 100 # $t0 = $t0 + 100 ->  $s = a + b + 100$`

`sub $t0, $t0, $t3 # $t0 = $t0 - $t3 ->  $s = a + b + 100 - c$`



7.4.2 Instructions logiques

Instruction	Signification	Description
and \$s1, \$s2, \$s3	$\$s1 = \$s2 \& \$s3$	ET bit-à-bit : opérandes dans 2 registres
andi \$s1, \$s2, 20	$\$s1 = \$s2 \& 20$	ET bit-à-bit : registre avec constante
or \$s1, \$s2, \$s3	$\$s1 = \$s2 \mid \$s3$	OU bit-à-bit : opérandes dans 2 registres
ori \$s1, \$s2, 20	$\$s1 = \$s2 \mid 20$	OU bit-à-bit : registre avec constante
nor \$s1, \$s2, \$s3	$\$s1 = \sim(\$s2 \mid \$s3)$	NOR bit-à-bit : opérandes dans 2 registres

Exemple de ET logique

0110100110100011 A

1010101010101010 B

0010100010100010 A and B



7.4.3 Instructions de décalage

Instruction	Signification	Description
<code>sll \$s1, \$s2, 10</code>	<code>\$s1 = \$s2 << 10</code>	Décalage logique à gauche, quantité constante
<code>sllv \$s1, \$s2, \$s3</code>	<code>\$s1 = \$s2 << \$s3</code>	Décalage logique à gauche, quantité dans registre
<code>srl \$s1, \$s2, 10</code>	<code>\$s1 = \$s2 >>> 10</code>	Décalage logique à droite, quantité constante
<code>srlv \$s1, \$s2, \$s3</code>	<code>\$s1 = \$s2 >>> \$s3</code>	Décalage logique à droite, quantité dans registre
<code>sra \$s1, \$s2, 10</code>	<code>\$s1 = \$s2 >> 10</code>	Décalage arithmétique à droite, quantité constante
<code>srav \$s1, \$s2, \$s3</code>	<code>\$s1 = \$s2 >> \$s3</code>	Décalage arithmétique à droite, quantité dans registre

- Décalage logique (sll / srl) : introduit toujours un 0
 - `11010011 << 2 → 01001100`
 - `11010011 >>> 2 → 00110100`
- Décalage arithmétique (sra) : ext. de signe (0 ou 1)
 - `11010011 >> 2 → 11110100`

7.4.4 Instructions de transfert

Instruction	Signification	Description
lw \$s1, 20(\$s2)	$\$s1 = M[\$s2 + 20]$	Charger un mot (32 bits)
lh \$s1, 20(\$s2)	$\$s1 = M[\$s2 + 20]$	Charger un demi-mot (16 bits) + extension du signe
lhu \$s1, 20(\$s2)	$\$s1 = M[\$s2 + 20]$	Charger un demi-mot (16 bits) + 0 dans bits de poids fort
lb \$s1, 20(\$s2)	$\$s1 = M[\$s2 + 20]$	Charger un octet (8 bits) + extension du signe
lbu \$s1, 20(\$s2)	$\$s1 = M[\$s2 + 20]$	Charger un octet (8 bits) + 0 dans bits de poids fort
lui \$s1, 20	$\$s1 = 20 * 2^{16}$	Constante dans les 16 bits de poids fort
sw \$s1, 20(\$s2)	$M[\$s2 + 20] = \$s1$	Stocker un mot (32 bits)
sh \$s1, 20(\$s2)	$M[\$s2 + 20] = \$s1$	Stocker un demi-mot (16 bits)
sb \$s1, 20(\$s2)	$M[\$s2 + 20] = \$s1$	Stocker un octet (8 bits)

- Load : on **charge** une donnée dans un registre ($M \rightarrow R$)
- Store : on **sauvegarde** la donnée du registre ($R \rightarrow M$)
- w = word (32), h = half-word (16), b = byte (8)
- **Adressage indexé** : $20(\$s2) \rightarrow \$s2 + 20$



7.4.5 Pseudo-instructions

- **Instructions supplémentaires** proposées par l'assembleur MIPS
- **Faciles à remplacer** par une ou plusieurs instructions de base

Exemples

move \$t2, \$t1 → add \$t2, \$zero, \$t1

li \$t1, 0x1234ABCD → lui **\$at**, 0x00001234
ori \$t1, \$at, 0x0000ABCD



7.4.6 Instructions de branchement

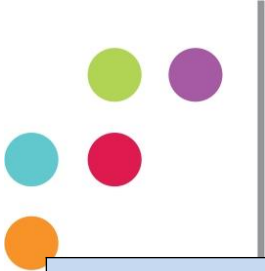
- **Branchement conditionnel** : détermine l'adresse de l'**instruction suivante** sur base d'une condition
if « condition » go to « instruction suivante »
- L'instruction suivante est désignée par une **étiquette**

Exemple de branchement « si ... alors ... »

Java	Assembleur
<pre>if (n > 0) n = n - a; n = n + 1;</pre>	<pre>blez \$t0, SUITE sub \$t0, t0, \$t1 SUITE: addi \$t0, \$t0, 1</pre>

SUITE →

<!> Inversion de la condition <!> → branchement si (n <= 0)



7.4.6 Instructions de branchement

Instruction	Signification	Description
<code>beq \$s1, \$s2, Label</code>	if ($\$s1 == \$s2$) go to <i>Label</i>	Branchement vers <i>label</i> en cas d'égalité des deux registres
<code>bne \$s1, \$s2, Label</code>	if ($\$s1 != \$s2$) go to <i>Label</i>	Branchement vers <i>label</i> en cas d'inégalité des deux registres
<code>bgez \$s1, Label</code>	if ($\$s1 \geq 0$) go to <i>Label</i>	Branchement vers <i>label</i> si registre supérieur ou égal à zéro
<code>bgtz \$s1, Label</code>	if ($\$s1 > 0$) go to <i>Label</i>	Branchement vers <i>label</i> si registre strictement supérieur à zéro
<code>blez \$s1, Label</code>	if ($\$s1 \leq 0$) go to <i>Label</i>	Branchement vers <i>label</i> si registre inférieur ou égal à zéro
<code>bltz \$s1, Label</code>	if ($\$s1 < 0$) go to <i>Label</i>	Branchement vers <i>label</i> si registre strictement inférieur à zéro
<code>bgezal \$s1, Label</code>	if ($\$s1 \geq 0$) $\$ra = PC + 4$; go to <i>Label</i> (branch and link)	Appel du sous-programme <i>label</i> si registre supérieur ou égal à zéro ; l'adresse de retour est placée dans le registre $\$ra$
<code>bltzal \$s1, Label</code>	if ($\$s1 < 0$) $\$ra = PC + 4$; go to <i>Label</i> (branch and link)	Appel du sous-programme <i>label</i> si registre strictement inférieur à zéro ; l'adresse de retour est placée dans le registre $\$ra$

- Appel conditionnel d'un sous-programme
 - Instructions `bgezal` et `bltzal` → « **branch and link** »
 - **Branchement** vers le SP si la condition est satisfaite
 - Adresse de l'instruction qui suit le branchement placée dans le registre $\$ra$ (**return address**)



7.4.6 Instructions de branchement

● Branchement inconditionnel ou saut

Instruction	Signification	Description
j <i>label</i>	go to <i>label</i>	Saut vers l'instruction <i>label</i>
jr \$ra	go to \$ra	Saut vers l'instruction dont l'adresse est précisée dans le registre. \$ra est souvent utilisé dans le cas d'un retour de sous-programme.
jal <i>label</i>	\$ra=PC+4; go to <i>label</i>	Appel du sous-programme <i>label</i> ; l'adresse de retour est placée dans le registre \$ra
jalr \$s1	\$ra=PC+4; go to \$s1	Appel du sous-programme dont l'adresse est dans \$s1 ; l'adresse de retour est placée dans le registre \$ra
jalr \$s1, \$s2	\$s1=PC+4; go to \$s2	Appel du sous-programme dont l'adresse est dans \$s2 ; l'adresse de retour est placée dans le registre \$s1

● Appels de sous-programmes

- Instructions jal et jalr → « **jump and link** »
- **Branchement** vers le sous-programme
- Adresse de l'instruction qui suit le saut placée dans le registre \$ra (**return address**)

7.4.6 Instructions de branchement

Exemple de « si ... alors ... sinon ... »

Rappel : inversion de la condition !

	Java	Assembleur
ELSE	if (n > 0) n = n - a; else n = n + b;	blez \$t0, ELSE sub \$t0, t0, \$t1 j SUITE
SUITE	n = n + 1;	ELSE: add \$t0, \$t0, \$t2 SUITE: addi \$t0, \$t0, 1

Exemple de « si ... alors ... sinon si ... alors ... sinon ... »

	Java	Assembleur
ELIF	if (n > 0) n = n - a;	blez \$t0, ELIF sub \$t0, t0, \$t1 j SUITE
ELSE	else if (n < 0) n = n + a;	ELIF: bgez \$t0, ELSE add \$t0, t0, \$t1 j SUITE
SUITE	else n = a; n = n + 1;	ELSE: add \$t0, \$zero, \$t1 SUITE: addi \$t0, \$t0, 1

7.4.6 Instructions de branchement

Exemple de boucle

	Java	Assembleur
WHILE	<pre>s = 0; n = 100; while (n > 0) { s = s + n; n = n - 1; }</pre>	<pre>li \$t0, 0 li \$t1, 100 WHILE: blez \$t1, END add \$t0, \$t0, \$t1 subi \$t1, \$t1, 1 j WHILE END: nop</pre>
END		

- L'instruction **subi** est une **pseudo-instruction**

`subi $t1, $t1, 1` → `addi $at, $zero, 1`
`sub $t1, $t1, $at`



CH07 - Programmation en assembleur

7.1 INTRODUCTION

7.2 CARACTÉRISTIQUES DU PROCESSEUR MIPS

7.3 REGISTRES DU PROCESSEUR

7.4 FAMILLES D'INSTRUCTIONS

7.5 MODES D'ADRESSAGE

7.5.1 INTRODUCTION

7.5.2 MODES D'ADRESSAGE DES BRANCHEMENTS

7.5.3 MODES D'ADRESSAGE DES DONNÉES

[...]



7.5 Modes d'adressage

7.5.1 Introduction

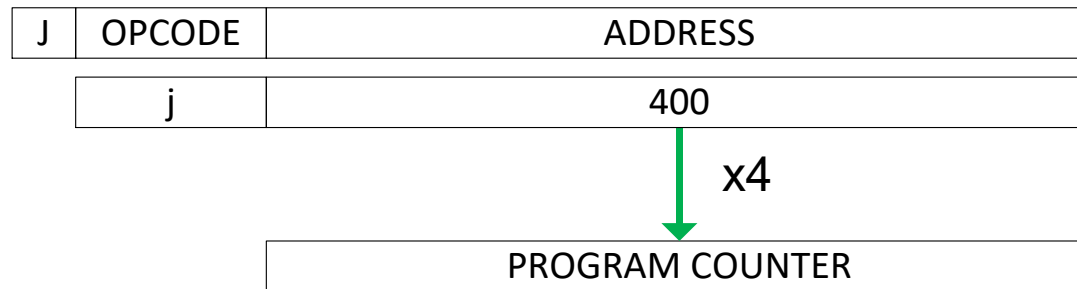
- On peut avoir besoin de calculer une adresse pour déterminer :
 - la **prochaine instruction** à exécuter (branchement et saut), ou
 - l'**emplacement d'une donnée**
- **mode d'adressage** = méthode pour **calculer la valeur d'une adresse en mémoire** à partir
 - des opérandes d'une instruction (adresse, constante, numéro de registre)
 - du contenu d'un registre spécial (PC)



7.5.2 Modes d'adressage des branchements

- But : obtenir l'adresse de l'instruction suivante
- **Adressage absolu** – adresse spécifiée dans l'instruction

j 400 # saut vers l'adresse 1600 (400×4)



➤ En pratique : j label



- ```
beq $t1,$t2,25 # if ($t1==$t2) branch to
 (PC+4)+100 (100 = 25*4)
```

```

graph TD
 PC[PROGRAM COUNTER] --> ADD[ADDITION]
 ADD --> ALU[ALU]
 ALU --> PC
 style ALU fill:#ffff00,stroke:#000,stroke-width:2px
 style PC fill:#fff,stroke:#000,stroke-width:1px
 style ADD fill:#fff,stroke:#000,stroke-width:1px

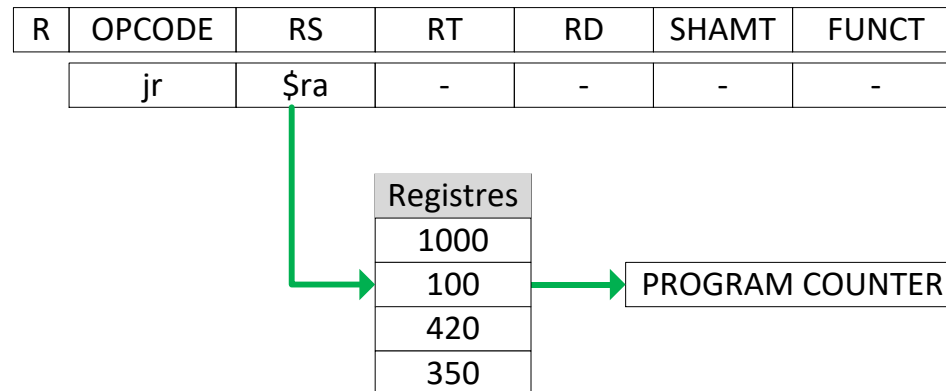
```



## 7.5.2 Modes d'adressage des branchements

- **Adressage indirect** – adresse dans 1 registre

`jr $ra # saut vers l'adresse dans $ra`





## 7.5.3 Modes d'adressage des données

- **Adressage immédiat** – donnée présente « immédiatement » dans l'instruction

```
addi $s1, $s2, 150 # $s1=$s2+150
```

- **Adressage implicite** – donnée dans un registre prédéfini par le processeur

```
mult $s1, $s2 # {Hi,Lo} = $s1*$s2
```



## 7.5.3 Modes d'adressage des données

- **Adressage indexé** – adresse effective = adresse de base dans registre + décalage

`lw $s1, 40($s2) # $s1 = MEM[$s2+40]`

| Registre | Contenu  |   | Adresse | Contenu |
|----------|----------|---|---------|---------|
| \$s1     | 0        |   | 400     | 20      |
| \$s2     | 360 + 40 | ↗ | ...     | ...     |
| ...      | ...      |   | 1500    | 400     |

➤ Au final, \$s1 contiendra 20



## 7.5.3 Modes d'adressage des données

- **Adressage direct** – l'instruction indique le registre qui contient la donnée ou son adresse en mémoire

```
add $s1,$s2,$s3 # $s1 = $s2 + $s3
```

```
lw $s1, 1500 # $s1 = MEM[1500] <!> pseudo
```

```
→ lw $s1, 1500($zero)
```

- **Adressage indirect** – l'instruction indique le registre où se trouve l'adresse de la donnée

```
lw $s1, ($s2) # $s1 = MEM[$s2] <!> pseudo
```

```
→ lw $s1, 0($s2)
```



# CH07 - Programmation en assembleur

[...]

## 7.5 MODES D'ADRESSAGE

## 7.6 APPELS DE SOUS-PROGRAMMES

### 7.6.1 PRINCIPE DE BASE

### 7.6.2 PARAMÈTRES ET VALEURS DE RETOUR

### 7.6.3 SAUVEGARDE DES REG. ET DE L'ADRESSE DE RETOUR

### 7.6.4 EXEMPLE DE SOUS-PROGRAMME

## 7.7 APPELS SYSTÈME

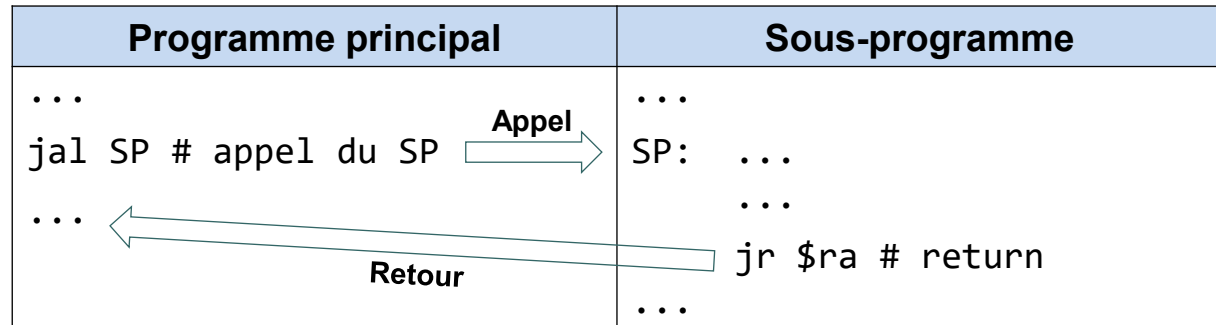
## 7.8 DIRECTIVES

## 7.9 EXEMPLES DE PROGRAMMES

## 7.6 Appels de sous-programmes

### 7.6.1 Principe de base

- Utilisation des instructions de type
  - « Branch and link » : `bgezal`, `bltzal`
  - « Jump and link » : `jal`, `jalr`
- **Branchement** / saut vers la première instruction du sous-programme
- Sauvegarde de l'**adresse de retour** dans `$ra`





## 7.6.2 Paramètres et valeurs de retour

- Paramètres transmis via les registres \$a0 à \$a3
- Résultats renvoyés via les registres \$v0 et \$v1
- Types
  - Valeurs numériques sur 32 bits
  - Adresse de début d'une chaîne ou d'un tableau
  - Longueur d'une chaîne ou d'un tableau
- Si le nombre de paramètres ou de valeurs renvoyées est supérieur...
  - Utilisation d'une pile en mémoire



## 7.6.2 Paramètres et valeurs de retour

### Exemple : MIN(a, b)

```
MAIN: li $a0, 23 # on charge la valeur 23 dans $a0
 li $a1, 7 # on charge la valeur 7 dans $a1
 jal MIN # on appelle le sous-programme
 ... # suite du programme, $v0 contient
 # le résultat de min(23,7)

MIN: move $v0, $a0 # on suppose que $a0 (a) est
 # le minimum
 ble $a0, $a1, FIN # si $a0 <= $a1, alors a est
 # bien le minimum
 move $v0, $a1 # sinon, le minimum est $a1 (b)

FIN: jr $ra # return
```



## 7.6.3 Sauvegarde des registres et de l'adresse de retour

- En assembleur, pas de notion de variable locale
- Un sous-programme peut modifier des registres qui étaient utilisés par le programme appelant !
- Convention de sauvegarde:
  1. Le programme **appelant** sauvegarde si nécessaire les registres \$t0 à \$t9, \$a0 à \$a3 et \$v0 à \$v1
  2. Le sous-programme **appelé** sauvegarde les registres \$s0 à \$s7 qu'il modifie, ainsi que le registre \$ra s'il effectue un **appel de SP en cascade**



## 7.6.3 Sauvegarde des registres et de l'adresse de retour

- Utilisation d'une **zone mémoire** pour créer une **pile**
- Généralement : pile située en « haut » de la mémoire et éléments **empilés vers le bas** (adresses décroissantes)
- Le registre `$sp` (***stack pointer***) contient l'adresse du **sommet** de la pile
- Opérations « push » et « pop » pour empiler / dépiler

```
push $s0 ⇒ addi $sp, $sp, -4 # ajuste le pointeur de pile (-4)
 sw $s0, 0($sp) # on place $s0 au sommet de la pile
```

```
pop $s0 ⇒ lw $s0, 0($sp) # on récupère la valeur de $s0
 addi $sp, $sp, 4 # ajuste le pointeur de pile (+4)
```

## 7.6.3 Sauvegarde des registres et de l'adresse de retour

### Exemple



Contexte : B a été appelé par A, il utilise les registres `$s0` et `$s1` et appelle à son tour le sous-programme C

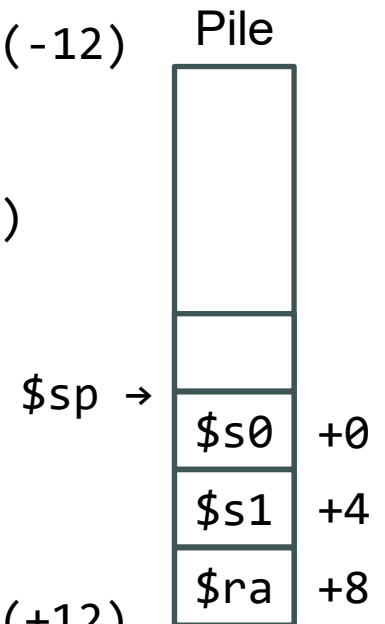
```
B: addi $sp, $sp, -12 # ajuster pointeur de pile (-12)
 sw $s0, 0($sp) # sauvegarder $s0
 sw $s1, 4($sp) # sauvegarder $s1
 sw $ra, 8($sp) # sauvegarder $ra (retour A)
```

...

```
jal C # appeler sous-programme C
```

...

```
lw $s0, 0($sp) # restaurer $s0
lw $s1, 4($sp) # restaurer $s1
lw $ra, 8($sp) # restaurer $ra (retour A)
addi $sp, $sp, 12 # ajuster pointeur de pile (+12)
jr $ra # retour vers programme A
```





## 7.6.4 Exemple : $\min(a, b, c)$

### Exemple : soit la fonction $\min(a, b)$

Les valeurs de  $a$  et  $b$  sont transmises dans  $\$a0$  et  $\$a1$ , le résultat est renvoyé dans  $\$v0$

```
MIN: move $v0, $a0 # on suppose que $a0 (a) est le min
 sub $t0, $a0, $a1 # $t0 = $a0 - $a1 (= a - b)
 blez $t0, FIN # si $t0 <= 0, a est bien le minimum
 move $v0, $a1 # sinon, le minimum est $a1 (b)
FIN: jr $ra
```

Calculons à présent  $\min(a, b, c) = \min(\min(a, b), c)$

Exemple numérique :  $a=23$ ,  $b=17$ ,  $c=42$



## 7.6.4 Exemple : min(a, b, c)

|                         |                       |                                  |                                |
|-------------------------|-----------------------|----------------------------------|--------------------------------|
|                         | MAIN:li \$t0, 23      | # \$t0 = a = 23                  | } Initialisation de<br>a, b, c |
|                         | li \$t1, 17           | # \$t1 = b = 17                  |                                |
|                         | li \$t2, 42           | # \$t2 = c = 42                  |                                |
| Sauvegarde<br>de \$t2   | { addi \$sp, \$sp, -4 | # ajuster pointeur de pile (-4)  |                                |
|                         | sw \$t2, 0(\$sp)      | # sauvegarder \$t2               |                                |
| Appel de<br>min(23,17)  | { move \$a0, \$t0     | # \$a0 = \$t0 [a]                |                                |
|                         | move \$a1, \$t1       | # \$a1 = \$t1 [b]                |                                |
|                         | jal MIN               | # calcul de min(23,17) = 17      |                                |
| Restauration<br>de \$t2 | { lw \$t2, 0(\$sp)    | # restaurer \$t2                 |                                |
|                         | addi \$sp, \$sp, 4    | # ajuster pointeur de pile (+4)  |                                |
| Appel de<br>min(17,42)  | { move \$a0, \$v0     | # \$a0 = \$v0 = min(23,17) [a]   |                                |
|                         | move \$a1, \$t2       | # \$a1 = \$t2 [b]                |                                |
|                         | jal MIN               | # calcul de min(17,42) = 17      |                                |
|                         | ...                   | # \$v0 = 42 = résultat du calcul |                                |

Note : dans l'absolu, la sauvegarde du registre \$t2 est inutile vu que le sous-programme min() ne le modifie pas



# CH07 - Programmation en assembleur

[...]

7.5 MODES D'ADRESSAGE

7.6 APPELS DE SOUS-PROGRAMMES

7.7 APPELS SYSTÈME

7.8 DIRECTIVES

7.9 EXEMPLES DE PROGRAMMES



## 7.7 Appels système

- Les **appels système** fournissent des services d'entrée/sortie

| Service                      | Code | Arguments                                                                     | Result                       |
|------------------------------|------|-------------------------------------------------------------------------------|------------------------------|
| print integer                | 1    | \$a0 = integer to print                                                       |                              |
| print string                 | 4    | \$a0 = address of null-terminated string to print                             |                              |
| read integer                 | 5    |                                                                               | \$v0 contains integer read   |
| read string                  | 8    | \$a0 = address of input buffer<br>\$a1 = maximum number of characters to read |                              |
| exit (terminate execution)   | 10   |                                                                               |                              |
| print character              | 11   | \$a0 = character to print                                                     |                              |
| read character               | 12   |                                                                               | \$v0 contains character read |
| exit2 (terminate with value) | 17   | \$a0 = termination result                                                     |                              |

- Les registres du processeur **ne sont pas modifiés** sauf \$v0 si mentionné



## 7.7 Appels système

- Pour effectuer un appel système
  1. Placer le numéro du service souhaité dans `$v0`
  2. Placer les arguments éventuels dans `$a0`, `$a1`
  3. Exécuter l'instruction `syscall`
  4. Récupérer le résultat éventuel dans `$v0`

### Exemple : afficher une valeur entière

```
li $a0, 42 # $a0 = valeur à afficher
li $v0, 1 # $v0 = numéro de l'appel système
syscall # -> la valeur 42 est affichée en console
```



## 7.7 Appels système

### Exemple : saisie et affichage de nombres entiers

```
.data
INVITE: .asciiz "Introduisez un nombre entier : "
.text
LOOP: la $a0, INVITE # Affichage du message d'invite
 li $v0, 4
 syscall
 li $v0, 5 # Saisie d'un nombre entier
 syscall
 beq $v0, $zero, EXIT # Si N=0, on s'arrête
 move $a0, $v0 # Affichage du nombre
 li $v0, 1
 syscall
 li $a0, '\n' # Affichage retour à la ligne
 li $v0, 11
 syscall
 j LOOP # Saisie du nombre suivant
EXIT: li $v0, 10 # Fin du programme
 syscall
```



# CH07 - Programmation en assembleur

[...]

7.5 MODES D'ADRESSAGE

7.6 APPELS DE SOUS-PROGRAMMES

7.7 APPELS SYSTÈME

7.8 DIRECTIVES

7.9 EXEMPLES DE PROGRAMMES



## 7.8 Directives

- Une **directive** est un mot-clé qui commence par « . »
- Elle permet d'agir sur le processus d'assemblage et de définir des données littérales qui seront utilisées par le programme

| Directive | Description                                                                      |
|-----------|----------------------------------------------------------------------------------|
| .ascii    | Store the string in the Data segment but do not add null terminator              |
| .asciiz   | Store the string in the Data segment and add null terminator                     |
| .byte     | Store the listed value(s) as 8-bit bytes                                         |
| .data     | Subsequent items stored in Data segment at next available address                |
| .half     | Store the listed value(s) as 16-bit halfwords on halfword boundary               |
| .space    | Reserve the next specified number of bytes in Data segment                       |
| .text     | Subsequent items (instructions) stored in Text segment at next available address |
| .word     | Store the listed value(s) as 32-bit words on word boundary                       |

- D'autres directives existent (macro, include...)



## 7.8 Directives (exemple)

`.data`

A: `.word` 23

B: `.word` 17

`.text`

```
MAIN: lw $s0, A # Initialisation
 lw $s1, B
 move $a0, $s0 # Appel de min()
 move $a1, $s1
 jal MIN # $v0 = min(23,17)
 li $a0, $v0 # Affichage résultat
 li $v0, 1 # 1 = print integer
 syscall
 li $v0, 10 # 10 = exit
 syscall
```

(suite du code)

```
MIN: move $v0, $a0
 sub $t0, $a0, $a1
 blez $t0, FIN
 move $v0, $a1
FIN: jr $ra
```



# CH07 - Programmation en assembleur

[...]

7.5 MODES D'ADRESSAGE

7.6 APPELS DE SOUS-PROGRAMMES

7.7 APPELS SYSTÈME

7.8 DIRECTIVES

7.9 EXEMPLES DE PROGRAMMES



## 7.9 Exemples de programmes

### 7.9.1 La fonction « strcpy »

#### La fonction 'strcpy' en C

```
void strcpy(char x[], char y[]) {
 int i ;
 i = 0 ;
 while ((x[i] = y[i]) != '\0') {
 i = i + 1;
 };
}
```

On copie les caractères de y vers x jusqu'au  
caractère de fin de chaîne '\0' (inclus)

y   

|   |   |   |   |   |    |
|---|---|---|---|---|----|
| H | E | L | L | O | \0 |
|---|---|---|---|---|----|

 → x   

|   |   |   |   |   |    |
|---|---|---|---|---|----|
| H | E | L | L | O | \0 |
|---|---|---|---|---|----|



## 7.9.1 La fonction « strcpy »

### ‘strcpy’ en langage d’assemblage

```
SUB: add $t0,$zero,$zero # $t0 = 0 + 0 | i = 0
L1: add $t1,$a1,$t0 # $t1 = $a1 + $t0 | t1 = adr(y[0])+i = adr(y[i])
 lbu $t2, 0($t1) # $t2 = M[$t1+0] | t2 = y[i]
 add $t3, $a0, $t0 # $t3 = $a0 + $t0 | t3 = adr(x[0])+i = adr(x[i])
 sb $t2, 0($t3) # M[$t3+0] = $t2 | x[i] = t2
 beq $t2, $zero, L2 # si ($t2 == 0) aller à L2
 addi $t0, $t0, 1 # $t0 = $t0 + 1 | i = i + 1
 j L1 # aller à L1
L2: jr $ra # fin du strcpy
```

$\$a0$  contient la référence vers  $x[]$  et  $\$a1$  la référence vers  $y[]$ ,  
 $\$t0$  joue le rôle de la variable  $i$ ,  $\$t2$  mémorise le caractère courant.



## 7.9.2 La suite de Fibonacci

Suite de Fibonacci : 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144...

|                                        |   |                                                  |                        |
|----------------------------------------|---|--------------------------------------------------|------------------------|
| Initialisation                         | { | final int N = 12; // Nombre de termes à calculer |                        |
| Définition des<br>2 premiers<br>termes | { | int t0 = N;                                      |                        |
|                                        |   | int t1 = 0;                                      |                        |
|                                        |   | System.out.println(t1);                          | } Affichage d'un terme |
|                                        |   | t0 = t0 - 1;                                     |                        |
| Calcul des<br>termes suivants          | { | int t2 = 1;                                      |                        |
|                                        |   | System.out.println(t2);                          |                        |
|                                        |   | t0 = t0 - 1;                                     |                        |
|                                        |   | while (t0 > 0) {                                 |                        |
|                                        |   | t3 = t1 + t2;                                    |                        |
|                                        |   | System.out.println(t3);                          |                        |
|                                        |   | t1 = t2;                                         |                        |
|                                        |   | t2 = t3;                                         |                        |
|                                        |   | t0 = t0 - 1;                                     |                        |
|                                        |   | }                                                |                        |
|                                        |   | exit(0);                                         | } Fin du programme     |



## 7.9.2 Exemple avancé (suite)

Version en langage d'assemblage

# Données

.data

N: .word 12 # Nombre de termes à calculer

# Programme

.text

lw \$t0, N # On initialise \$t0 avec N

add \$t1, \$zero, \$zero # On initialise le premier terme

move \$a0, \$t1 # On charge la valeur dans \$a0

jal PRINT # Appel sous-programme : print(\$a0)

addi \$t0, \$t0, -1 # On décrémente \$t0

PSEUDO

li \$t2, 1 # On initialise le deuxième terme

move \$a0, \$t2 # On charge la valeur dans \$a0

jal PRINT # Appel sous-programme : print(\$a0)

addi \$t0, \$t0, -1 # On décrémente \$t0



## 7.9.2 Exemple avancé (suite)

```
LOOP: blez $t0, END # Si $t0 <= 0 , on a fini
 add $t3, $t1, $t2 # Calcul terme suivant : $t3=$t1+$t2
 move $a0, $t3 # On charge la valeur dans $a0
 jal PRINT # Appel sous-programme : print($a0)
 move $t1, $t2 # On décale les termes
 move $t2, $t3 #
 addi $t0, $t0, -1 # On décrémente $t0
 j LOOP #
END: li $v0, 10 # Appel système 10 = exit
 syscall # -> Fin du programme

Sous-programme d'impression d'un terme de la suite
PRINT: li $v0, 1 # Appel système 1 = print integer
 syscall # -> Affichage de la valeur dans $a0
 li $a0, '\n' # Charge code ASCII '\n' dans $a0
 li $v0, 11 # Appel système 11 = print char
 syscall # -> Affichage retour à la ligne
 jr $ra # return
```



# CH07 - Programmation en assembleur

[...]

7.5 MODES D'ADRESSAGE

7.6 APPELS DE SOUS-PROGRAMMES

7.7 APPELS SYSTÈME

7.8 DIRECTIVES

7.9 EXEMPLES DE PROGRAMMES



# Exercices – Les bases

1. Additionner 123456 et 987654 et afficher le résultat
2. Idem ex1 en déclarant des données statiques A et B au moyen de la directive ' .word '
3. Idem ex2 en déclarant et en appelant un sous-programme PR\_INT qui affiche un nombre entier



# Exercices – Chaînes de caractères

1. Afficher les codes ascii<sup>(\*)</sup> des 3 premières lettres d'un mot défini avec la directive `'.ascii'`
2. Afficher les codes ascii des toutes les lettres d'un mot défini avec `'.ascii' + longueur`
3. Idem ex2, mais chaîne délimitée par `'\0'` (`'.asciiz'`)

(\*) Suggestion : réutiliser le `SP PR_INT`



# Exercices – Tableaux de nombres

1. Afficher les 3 premières valeurs d'un tableau de INT32 défini avec `'.word'`
2. Calculer et afficher la somme des valeurs d'un tableau de INT16 défini avec `'.half'` + longueur
3. Calculer, mémoriser et afficher les **sommes cumulées** des valeurs d'un tableau de INT32

Ex: [1, 2, 3, 4, 5] → [1, 3, 6, 10, 15]

Allouer un tableau de résultats avec `'.word 0:N'`

Définir un sous-programme qui affiche le contenu d'un tableau